Practical:01	Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort
__/___/2026	

a) Bubble Sort

Objective:

- To arrange elements in ascending order.
- To compare and swap adjacent elements.
- To understand its time complexity.

Program:

```
#include  <iostream>
using namespace std;
int main()
{
   int arr[] = {64, 34, 25, 12, 22, 11, 90};
   int n = sizeof(arr) / sizeof(arr[0]);
   cout << "Original Array: ";
   for (int i = 0; i < n; i++)
      cout << arr[i] << " ";
   for (int i = 0; i < n - 1; i++)
   {
      for (int j = 0; j < n - i - 1; j++)
      {
         if (arr[j] > arr[j + 1])
         {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
         }
      }
   }
   cout << "\nSorted Array: ";
   for (int i = 0; i < n; i++)
```

```
        cout << arr[i] << " ";
    return 0;
}
```

Output:

```
Original Array: 64 34 25 12 22 11 90
Sorted Array: 11 12 22 25 34 64 90
```

b) Selection Sort

Objective:

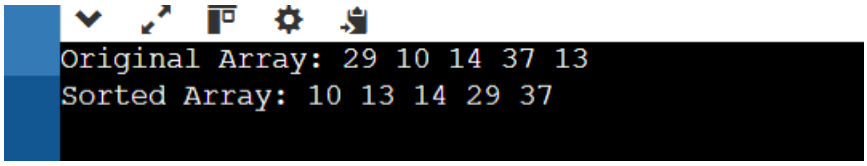
- To arrange elements in ascending order.
- To find the smallest element and place it correctly.
- To understand its time complexity.

Program:

```
#include <iostream>
using namespace std;
// Function to print the array
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
int main()
{
    int arr[] = {29, 10, 14, 37, 13};
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << "Original Array: ";
    printArray(arr, n);
    // Selection Sort
    for (int i = 0; i < n - 1; i++)
    {
        int min = i;
```

```
        // Find the minimum element
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[min])
                min = j;
        }
        // Swap the minimum element with the current position
        int temp = arr[min];
        arr[min] = arr[i];
        arr[i] = temp;
    }
    cout << "Sorted Array: ";
    printArray(arr, n);
    return 0;
}
```

Output:



c) Insertion Short

Objective:

- To arrange elements in ascending order.
- To insert each element into its correct position.
- To understand its time complexity.

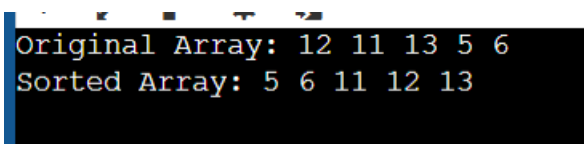
Program:

```
#include <iostream>
using namespace std;

// Function to print the array
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
```

```
        cout << arr[i] << " ";
    cout << endl;
}
int main()
{
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << "Original Array: ";
    printArray(arr, n);
    // Insertion Sort
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];  // Element to be inserted
        int j = i - 1;
        // Shift elements greater than key
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        // Insert key at the correct position
        arr[j + 1] = key;
    }
    cout << "Sorted Array: ";
    printArray(arr, n);
    return 0;
}
```

Output:



d) Merge Sort

Objective:

- To sort elements using the divide-and-conquer method.
- To divide and merge the elements.
- To understand its time complexity.

Program:

```
#include <iostream>

using namespace std;


void merge(int arr[], int low, int mid, int high)

{

   int i = low, j = mid + 1, k = 0;

   int temp[100];


   while (i <= mid && j <= high)

   {

      if (arr[i] < arr[j])

         temp[k++] = arr[i++];

      else

         temp[k++] = arr[j++];

   }


   while (i <= mid)

      temp[k++] = arr[i++];


   while (j <= high)

      temp[k++] = arr[j++];


   for (i = low, k = 0; i <= high; i++, k++)

      arr[i] = temp[k];

}
```

```
void mergeSort(int arr[], int low, int high)
{
   if (low < high)
   {
      int mid = (low + high) / 2;
      mergeSort(arr, low, mid);
      mergeSort(arr, mid + 1, high);
      merge(arr, low, mid, high);
   }
}
int main()
{
   int n, arr[100];
   cout << "Enter number of elements: ";
   cin >> n;
   cout << "Enter elements: ";
   for (int i = 0; i < n; i++)
      cin >> arr[i];
   mergeSort(arr, 0, n - 1);
   cout << "Sorted array: ";
   for (int i = 0; i < n; i++)
      cout << arr[i] << " ";
   return 0;
}
```

Output:

```
Enter number of elements: 5
Enter elements: 67 35 89 22 59
Sorted array: 22 35 59 67 89
```

e) Quick Sort

Objective:

- To sort elements using the divide-and-conquer method.
- To use a pivot for partitioning.
- To understand its time complexity.

Program:

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high)
{
   int pivot = arr[high];
   int i = low - 1;

   for (int j = low; j < high; j++)
   {
      if (arr[j] < pivot)
      {
         i++;
         swap(arr[i], arr[j]);
      }
   }
   swap(arr[i + 1], arr[high]);
   return i + 1;
}
void quickSort(int arr[], int low, int high)
{
   if (low < high)
   {
      int p = partition(arr, low, high);

      quickSort(arr, low, p - 1);
      quickSort(arr, p + 1, high);
   }
}
int main()
{
```

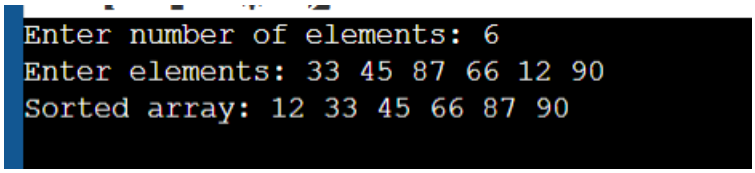
```
    int n, arr[100];

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    quickSort(arr, 0, n - 1);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

Output:



Analysis:

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

Conclusion:

Practical testing shows that elementary sorts like Bubble, Selection, and Insertion work fine for small datasets, but become extremely slow as data grows. For large datasets, advanced algorithms like Merge Sort and Quicksort are essential because they scale efficiently and execute much faster.